

Por que unificar os 4 modulos em um unico projeto?

Decisao arquitetural — Migracao para Nuxt 3 unificado

Contexto

O sistema Quanta Shop era composto por 4 projetos Replit separados e independentes:

Projeto	Tecnologia	Funcao
Site principal	Nuxt 3	Portal publico, vitrine de lojas
Agencia (afiliados)	Vue 2	Login, painel, area admin
Primeira Compra	React 18	Cadastro de clientes no caixa
API Backend	.NET 8	Backend compartilhado por todos

Por que unificar

1. Todos os modulos ja compartilhavam o mesmo backend.

A API .NET 8 era o ponto central de todos os 4 projetos. Qualquer atualizacao na API ja afetava todos simultaneamente — a separacao dos frontends nao evitava esse risco compartilhado.

2. Duplicacao de codigo e manutencao triplicada.

Autenticacao JWT, chamadas de API, estilos base e componentes de UI existiam em copias distintas. Um bug corrigido em um nao era corrigido nos outros, criando divergencias ao longo do tempo.

3. Vue 2 atingiu fim de vida em dezembro de 2023.

Manter o modulo de agencia em Vue 2 separado criava uma divida tecnica crescente. Uma migracao isolada seria mais custosa e arriscada do que integrar dentro do Nuxt 3 ja existente.

4. Replit penaliza projetos fragmentados.

Cada projeto separado tem seu proprio ciclo de deploy e banco de secrets. O secret `SQL_CONNECTION_STRING` precisava ser replicado nos 4 lugares — risco real de inconsistencia em producao.

Como o isolamento foi preservado — sem os riscos

A preocupacao sobre "comprometer um modulo durante manutencao de outro" e valida e foi tratada pela arquitetura interna do projeto unificado:

Isolamento por rotas: Cada modulo vive em seu proprio prefixo de URL (`/agencia/**`, `/primeira-compra/**`). Um erro numa pagina de agencia nao afeta o site principal.

Layouts independentes: Cada modulo tem seu proprio layout Vue (`agencia.vue`, `agencia-painel.vue`, `primeira-compra.vue`), sem heranca cruzada.

Estilos encapsulados: Cada modulo tem seu proprio arquivo SCSS (`agencia.scss`, `primeira-compra.scss`), sem vazamento de estilos entre modulos.

Middleware isolado: `agencia-auth.ts` e `agencia-admin.ts` protegem apenas as rotas de agencia, sem tocar no restante do site.

Componentes separados: `components/agencia/` e `components/primeira-compra/` sao completamente separados dos componentes do site principal.

Comparativo de riscos: separado vs. unificado

Cenário	Separado (antes)	Unificado (agora)
Atualizar a API .NET	Risco para todos os 4 projetos	Risco igual — sem diferença
Bug em componente da agência	Afeta so a agência	Afeta so a agência (rotas isoladas)
Atualizar dependencia do site	Risco de quebrar outros projetos	Controlado — uma unica versao de pacotes
Secret de banco de dados	4 pontos de exposicao	1 ponto de exposicao
Deploy fora do ar	Qualquer dos 4 pode estar fora	1 ponto unico de controle e monitoramento

Conclusao

A separacao original dava uma percepcao de isolamento, mas na pratica o risco compartilhado ja existia atraves da API comum. A unificacao reduz a superficie de manutencao, elimina duplicacao de codigo e permite uma estrategia de deploy mais segura — com a mesma fronteira de isolamento funcional que existia antes, agora garantida pela arquitetura de rotas e layouts do Nuxt 3.

Em resumo:
O projeto unificado se comporta como 3 mini-aplicacoes dentro do mesmo servidor Nuxt 3, compartilhando infraestrutura e codigo comum — com fronteiras de modulo mais rigidas e controladas do que a separacao por projetos Replit distintos permitia.